ICML
International Conference
On Machine Learning

A Deep Reinforcement Learning Perspective on Internet Congestion Control

Nathan Jay, Noga H. Rotman, P. Brighten Godfrey, Michael Schapira, Aviv Tamar

International Conference on Machine Learning (2019)

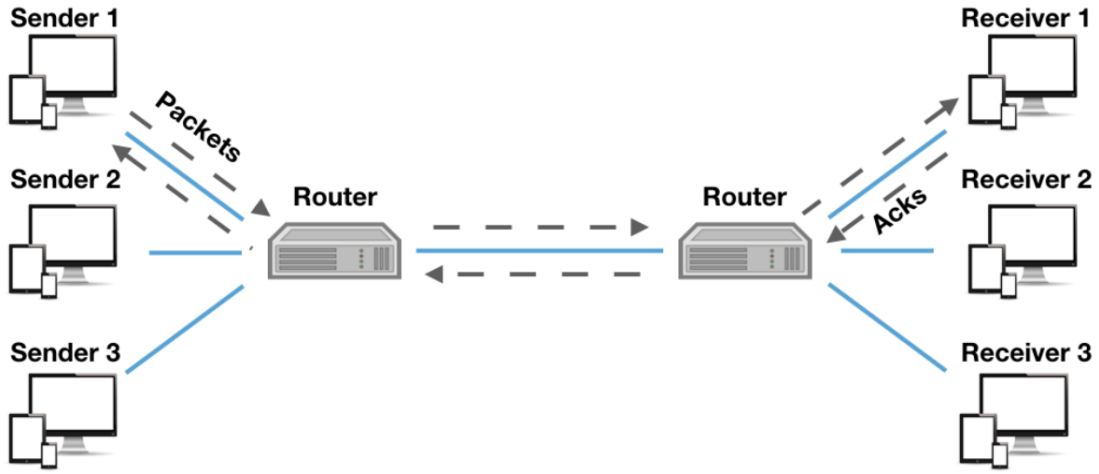
Elie Kfoury

Introduction

- In today's Internet, multiple network users contend over scarce communication resources
- Transmission rates of different traffic sources must be adapted dynamically
- This challenge is termed "**congestion control**"
- Congestion control has a crucial impact on user experience
  - Video streaming
  - Voice-over-IP,
  - Emerging Internet services such as AR/VR, IoT, edge computing, etc.

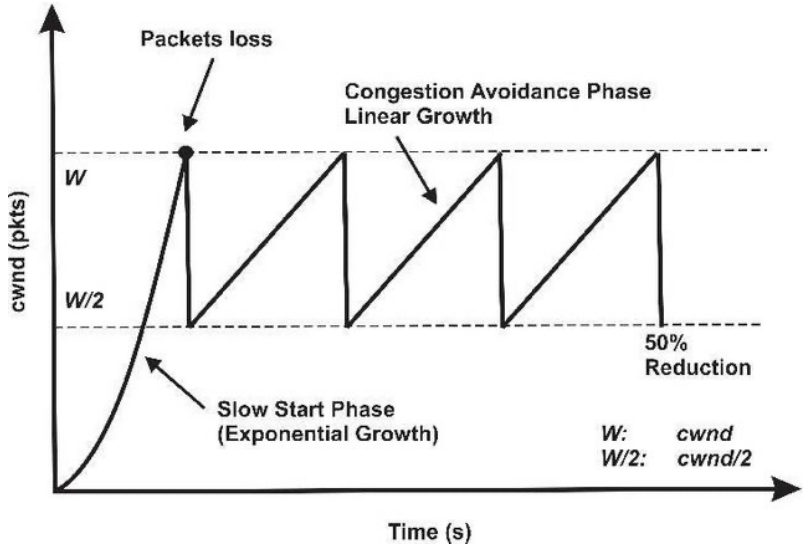
Congestion Control

- Flows sharing a single communication link
- The sender streams data packets to the receiver and constantly experiences feedback from the receiver
- The sender adjusts its transmission rate in response to this feedback
- Congestion is affected by the congestion control algorithm, link bandwidth, buffer size, round-trip-time, and many others



Congestion Control Algorithms

- Rates are selected in an uncoordinated, decentralized manner
- Packet loss is seen as a sign of congestion and results in decreasing the sending rate
- No explicit information about competing connections
  - Number
  - Times of entry/exit
  - Congestion control protocols
  - Link's bandwidth
  - Buffer size
  - Packet-queuing policy

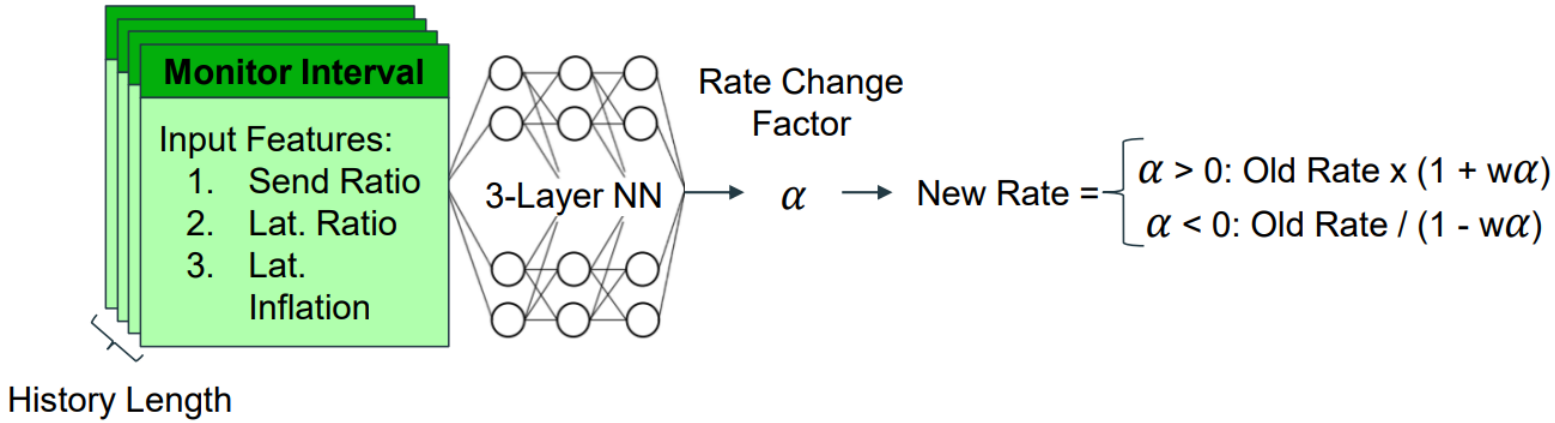


RL-Guided Congestion Control

- Locally-perceived history of feedback from the receiver reflects past traffic and network conditions
- Leveraged to determine the next choice of sending rate
- A fundamental deficiency of TCP is its innate inability to distinguish between congestion/non-congestion induced packet loss
  - Leads to highly suboptimal rate choices
- TCP is notoriously bad at adapting to variable network conditions

Aurora

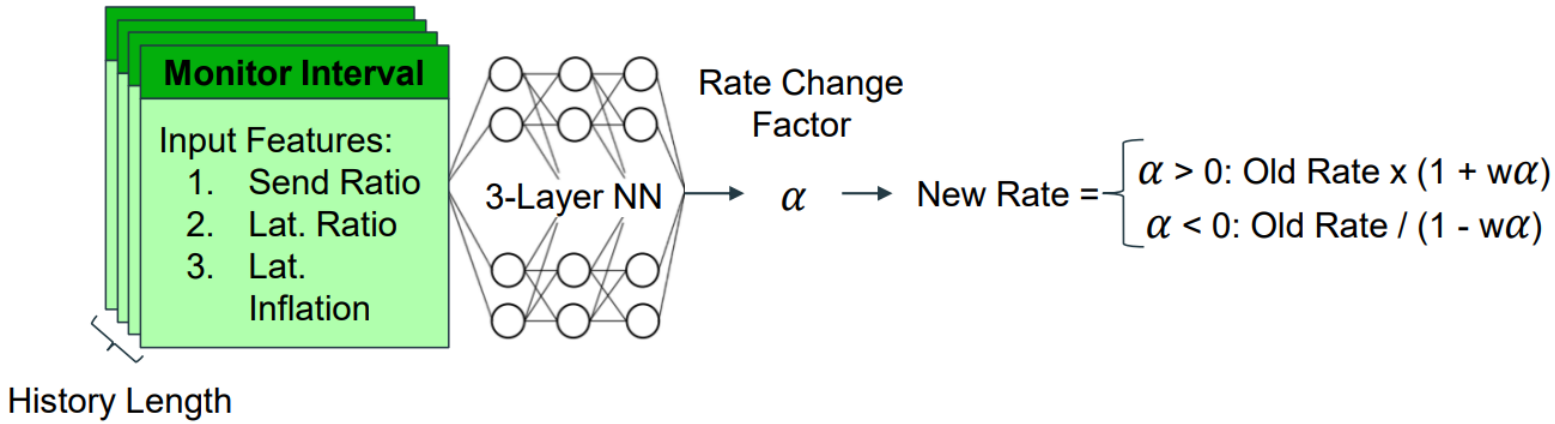
- Implementation of RL for congestion control
- The agent is the sender
- Actions translate to changes in sending rates
- States: latency (mean / min), sending ratio (packet sent / acknowledged), latency inflation



Aurora

- Fully connected neural network
- Two hidden layers composed of 32 → 16 neurons
- Reward function: rewards throughput while penalizing loss and latency

$$10 * throughput - 1000 * latency - 2000 * loss$$



Training and Testing

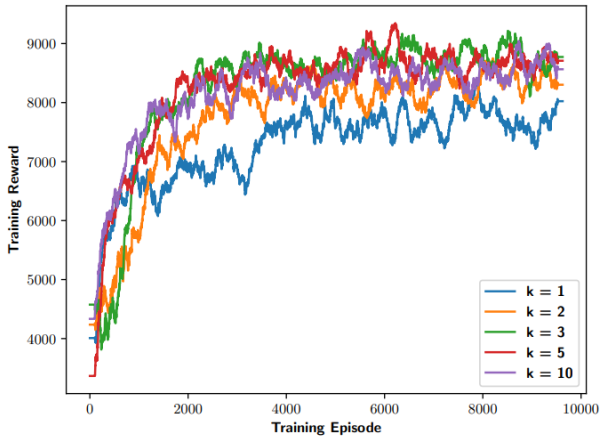
- Training on simulator

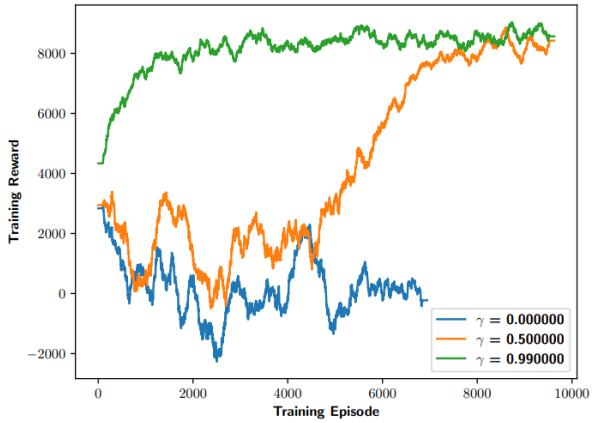
Capacity	Latency	Loss	Queue
1 - 6mbps	50 - 500ms	0 - 5%	1 - ~3000pkt

Testing on emulator

Capacity	Latency	Loss	Queue
1 - 128mbps	1 - 512ms	0 - 20%	1 - 10000pkt

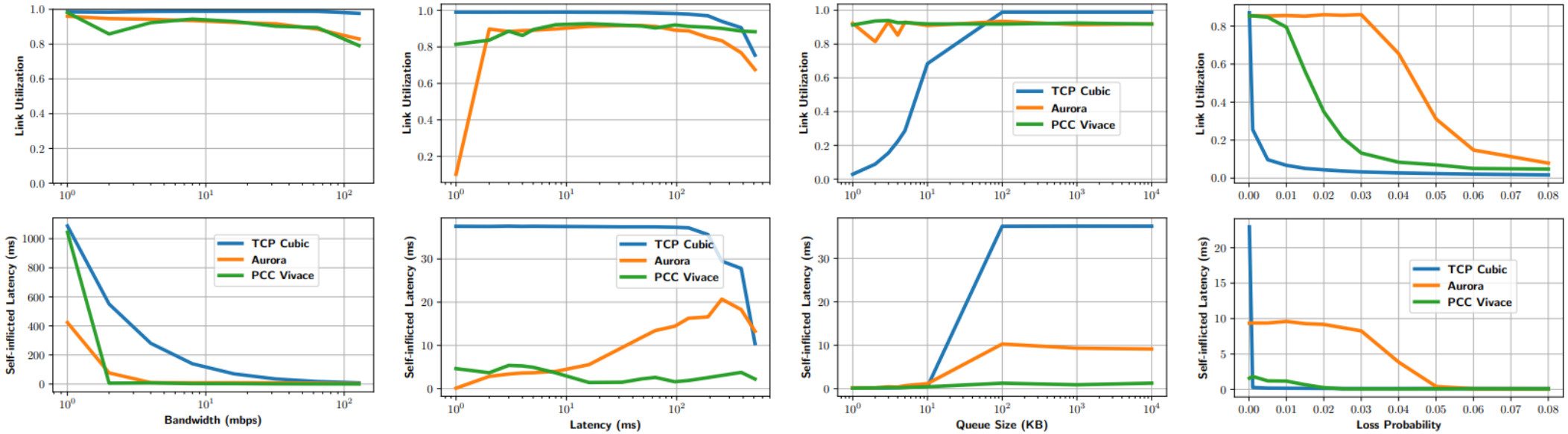
- A history length of *k* means that the agent makes a decision based on the *k* latest MIs worth of data





Results

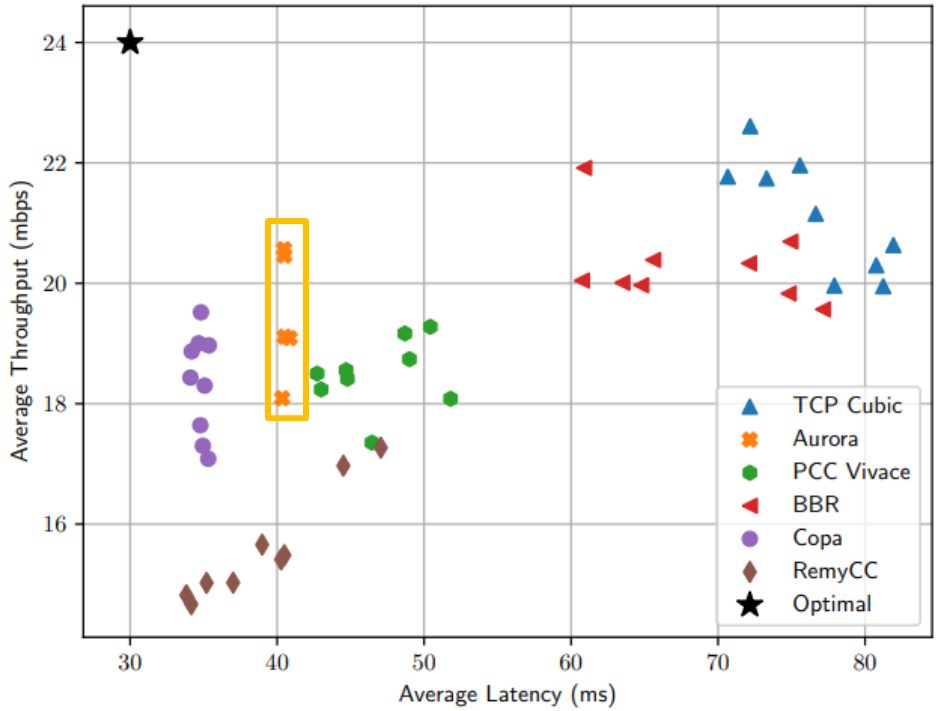
- Compared with TCP Cubic, PCC Vivace



(a) Bandwidth sensitivity (b) Latency sensitivity (c) Buffer size sensitivity (d) Loss rate sensitivity

Results

- Emulated dynamic link performance



Discussions and Future Work

- Simplistic attempt yet produced results comparable to state-of-the-art
- Limitations:
  - The tests are not thorough, and the parameters do not reflect today's Internet
  - Potential low fairness between flows
  - No consideration for the buffer size or queueing policies on the router
  - Limited number of measured information
- Future work:
  - Run the algorithm on datasets provided from production networks (e.g., CAIDA)
  - Investigate the fairness issue
  - Implementation as a kernel module

Thank You

Questions?